Formal Semantics for Agentic Tool Protocols: A Process Calculus Approach

Andreas Schlapbach
Swiss Federal Railways (SBB), SBB IT
Berne, Switzerland
schlpbch@gmail.com

The emergence of large language model agents capable of invoking external tools has created urgent need for formal verification of agent protocols. Two paradigms dominate this space: *Schema-Guided Dialogue* (SGD), a research framework for zero-shot API generalization, and the *Model Context Protocol* (MCP), an industry standard for agent–tool integration. While both enable dynamic service discovery through schema descriptions, their formal relationship remains unexplored. We present the first process calculus formalization of SGD and MCP, proving they are structurally bisimilar under a well-defined mapping Φ. We demonstrate that the reverse mapping Φ^{-1} is partial and lossy, revealing critical gaps in MCP's expressivity. Through bidirectional analysis, we identify four principles—semantic completeness, explicit action boundaries, failure mode documentation, and inter-tool relationship declaration—as necessary and sufficient conditions for full behavioral equivalence. We formalize these principles as type-system extensions MCP^+, proving $\text{MCP}^+ \cong \text{SGD}$. Our work provides the first formal foundation for verified agent systems and establishes schema quality as a provable safety property. Practically, this means that the current MCP specification has expressiveness gaps compared to SGD and would benefit from the proposed extensions.

1 Introduction

The rapid deployment of large language model (LLM) agents [20, 5] in production systems has created a critical gap: while these agents—capable of multi-step reasoning [22] and reactive tool use [24]—orchestrate complex workflows across services, we lack formal methods to verify their correctness, safety, and behavioral properties. Two paradigms have emerged to address agent–tool integration:

1.1 The Schema-Guided Dialogue Systems (SGD)

The SGD dataset [18] introduced a paradigm shift in task-oriented dialogue: rather than training on fixed ontologies, SGD models learn to interpret arbitrary API schemas at runtime. Key components are *intents* (high-level user goals), *slots* (parameters to collect), *schemas* (JSON definitions with natural-language descriptions), and *state tracking* (frame-based representation across turns). Dialogue state tracking has a longer history in shared tasks [23], and BERT-based encoders [8] enabled strong cross-service generalization. Fast and robust SGD trackers [11] demonstrated practical viability at scale. More recently, in-context learning has shown that few demonstration examples suffice to adapt to new schemas—mirroring the zero-shot generalization that SGD pioneered [4, 16].

1.2 Model Context Protocol (MCP)

The MCP is an open protocol that enables seamless integration between LLM applications and external data sources and tools. The architecture comprises a *host* (LLM environment), a *client* (protocol handler),

Submitted to:
Express/SOS 2026

and a *server*. Servers provide the fundamental building blocks for adding context to language models via MCP. These primitives enable rich interactions between clients, servers, and language models. It standardizes agent–tool communication through three primitives: Firstly, *Tools* enable models to interact with external systems, such as querying databases, calling APIs, or performing computations. Each tool is uniquely identified by a name and includes metadata describing its schema. Secondly, *Resources* allow servers to share data that provides context to language models, such as files, database schemas, or application-specific information. Each resource is uniquely identified by a URI. Thirdly, *Prompts* allow servers to provide structured messages and instructions for interacting with language models. Clients can discover available prompts, retrieve their contents, and provide arguments to customize them.

1.3 Shared Core Principles

Both approaches share a core principle: agents discover and reason about services through machine-readable schemas without retraining. This analysis is grounded in experience with an experimental ecosystem of over 10 agents. The agents are each a domain expert, following domain-driven design [21]. The agents are intelligently federated, coordinating through dynamically discovered schema relationships and the MCP rather than rigid orchestration logic. This federated architecture has revealed critical gaps between the theoretical elegance of SGD principles and the practical requirements of production agent systems: over 1,000 tool-to-tool dependencies must be managed, action boundaries must be enforced deterministically, and failure modes must be recoverable without human intervention.

The *formal* relationship between the two approaches remains unexplored. Can we prove they are equivalent? What properties are preserved under transformation? What gaps exist in either framework? This paper's aim is to provide answers to these questions by grounding the analysis in process calculus.

1.4 Contributions

This paper makes the following contributions:

1. **Formal semantics for** SGD **and** MCP using π-calculus, defining their syntax, operational semantics, and labeled transition systems (Section 3).
2. **Bisimulation proof** showing $\mathsf{SGD} \sim \mathsf{MCP}$ under mapping Φ, establishing structural and behavioral equivalence (Section 4).
3. **Gorla-Style analysis** of Φ showing that while it is a valid encoding it is neither surjective nor fully abstract (Section 5).
4. **Four principles as type system** formalizing necessary conditions for agent safety and schema quality. Safety can also be regarded as respecting a user's privacy by asking for permission before an action (Section 6).
5. **Extended calculus** MCP^+ proving that adding four principles creates full bijection: $\mathsf{MCP}^+ \cong \mathsf{SGD}$ (Section 7.3).

2 Background and Related Work

Consider a ticket booking agent which needs to book a train ticket. The agent needs first to check the user using its id. Then has has to validate if the user has a valid account and enough balance to book

the ticket. If the user has enough balance, the agent can book then book the ticket from an origin to a destination at a given date and travel class. The agent needs to send to print the ticket or send it via email.

If the agent is an AI agent, the questions requiring formal answers include: Needs the agent to *always* fetch the user? Can the third step of booking the ticket execute without the second step of checking the balance? What happens if the third step fails after debiting the account?

Current approaches to validate a complex ecosystems of agents rely on testing (incomplete coverage), prompt engineering [22] (no guarantees, brittle), or human review (not scalable). Agent frameworks grounded in BDI theory [17] offer structured reasoning, but stop short of protocol-level verification. Formal semantics enable *verification* (proving safety for all executions), *composition* (building complex systems from verified components), *optimization* (preserving equivalence under transformation), and *debugging* (tracing violations to specific protocol violations).

2.1 Process Calculi for Concurrency

Process calculi provide mathematical frameworks for reasoning about concurrent, communicating systems. The π-calculus [14] extends CCS [13] with mobile channels, enabling dynamic network topologies; essential for agent systems where tool availability changes at runtime.

Related work includes session types [12] for protocol verification, choreography calculi [6] for multiparty workflows, and probabilistic process algebras for stochastic systems [7]. However, all these frameworks assume static protocols. Agent systems require runtime schema interpretation. Work by [3] specifies temporal logic properties for a host agent architecture covering MCP and A2A. [15] take a different stance: they make the medium safe by construction through a type system; our work identifies the necessary and sufficient principles for such a type system to achieve full expressivity.

Highly relevant but somewhat orthogonal is the work on the LLMbda Calculus by [9] by providing a principled semantic foundation for reasoning about an agent's behavior and safety. The work addresses this gap by introducing an untyped call-by-value lambda calculus enriched with dynamic information-flow control and a small number of primitives for constructing prompt-response conversations. The calculus faithfully represents planner loops and their vulnerabilities, including the mechanisms by which prompt injection alters subsequent computation.

3 Process Calculus Formalization

In this paper we formalize SGD and MCP as communicating processes in π-calculus, defining their syntax, operational semantics, and observable behaviors. The π-calculus was chosen because: First, it makes topology dynamic where processes can acquire new channel names at runtime and, secondly, that we are accustomed to it through previous work on PICCOLA, a pure composition language based on this calculus [2] [1], [19].

3.1 SGD Process Calculus

The Schema-Guided Dialogue (SGD) dataset[1] consists of over 20k annotated multi-domain, task-oriented conversations between a human and a virtual assistant. These conversations involve interactions with services and APIs spanning 20 domains, such as banks, events, media, calendar, travel, and weather.

[1]The data set is publicly available at: https://github.com/google-research-datasets/dstc8-schema-guided-dialogue

Additionally, the dataset contains unseen domains and services in the evaluation set to quantify the performance in zero-shot or few-shot settings.

3.1.1 Syntax

The SGD data set has a well defined structure. Dialogues are represented as a list of turns, where each turn contains either a user or system utterance. The annotations for a turn are grouped into frames, where each frame corresponds to a single service. Each turn in the single domain dataset contains exactly one frame. In multi-domain datasets, some turns may have multiple frames.

Based on this structure we propose a possible grammar, where n is a name (string), d a natural-language description, R a list of required slots, O a list of optional slots, and $t \in \{\mathsf{true}, \mathsf{false}\}$ a transactionality flag:

$$
\begin{array}{lll}
S ::= & \mathsf{Intent}\langle n, d, R, O, t\rangle & \text{intent with metadata} \\
\mid & \mathsf{Slot}\langle \mathit{name}, \mathit{type}, \mathit{values}\rangle & \text{slot definition} \\
\mid & \mathsf{CollectSlot}\langle s, v\rangle\,.\,S & \text{slot collection, then } S \\
\mid & \mathsf{Execute}\langle \mathit{intent}, \mathit{bindings}\rangle & \text{API invocation} \\
\mid & \mathsf{Result}\langle \mathit{output}\rangle & \text{return value} \\
\mid & \mathsf{Error}\langle \mathit{type}, \mathit{msg}\rangle & \text{error state} \\
\mid & S_1 \mid S_2 & \text{parallel composition} \\
\mid & (\nu\, c)\; S & \text{channel restriction} \\
\mid & !S & \text{replication} \\
\mid & \mathbf{0} & \text{null process}
\end{array}
$$

3.1.2 Example: Train Booking in SGD

The example illustrates the intent to book a train, where origin, destination and date need to be set and a confirmation is required before booking.

```
BookTrainIntent = Intent<
  name = "BookTrain",
  description = "Books a train ticket",
  R = [Slot<"from", "string", ["8507000", "8508050"]>,
       Slot<"to", "string", ["8507000", "8508050"]>,
       Slot<"date", "date", []>],
  O = [Slot<"class", "string", ["economy", "business"]>],
  t = true  // Transaction requires confirmation
>
```

3.1.3 Operational Semantics

The transition rules for SGD processes are given in Figure 1.

$$\frac{\forall r \in R.\ r \in \mathrm{dom}(params)}{\mathsf{Intent}\langle n, d, R, O, t\rangle \xrightarrow{\mathsf{invoke}(n, params)} \mathsf{Execute}\langle n, params\rangle}\ \text{SGD-INVOKE-OK}$$

$$\frac{\exists r \in R.\ r \notin \mathrm{dom}(params)}{\mathsf{Intent}\langle n, d, R, O, t\rangle \xrightarrow{\mathsf{invoke}(n, params)} \mathsf{Error}\langle \mathsf{MissingSlots}, R \setminus params\rangle}\ \text{SGD-INVOKE-ERR}$$

$$\frac{}{\mathsf{CollectSlot}\langle s, v\rangle . S \xrightarrow{\mathsf{collect}(s,v)} S[s \mapsto v]}\ \text{SGD-COLLECT}$$

$$\frac{\begin{array}{c} t{=}\mathsf{true} \\ \mathsf{require\_approval} \\ output{=}\mathsf{invoke\_api}(n,b) \end{array}}{\mathsf{Execute}\langle n, b\rangle \xrightarrow{\mathsf{execute}} \mathsf{Result}\langle output\rangle}\ \text{SGD-EXECUTE-TX}$$

$$\frac{\begin{array}{c} t{=}\mathsf{false} \\ output{=}\mathsf{invoke\_api}(n,b) \end{array}}{\mathsf{Execute}\langle n, b\rangle \xrightarrow{\mathsf{execute}} \mathsf{Result}\langle output\rangle}\ \text{SGD-EXECUTE}$$

$$\frac{S_1 \xrightarrow{\alpha} S_1'}{S_1 \mid S_2 \xrightarrow{\alpha} S_1' \mid S_2}\ \text{SGD-PAR}$$

$$\frac{\begin{array}{c} S \xrightarrow{\alpha} S' \\ c \notin \alpha \end{array}}{(\nu\, c)\, S \xrightarrow{\alpha} (\nu\, c)\, S'}\ \text{SGD-RES}$$

Figure 1: Operational semantics for SGD processes.

3.2 MCP Process Calculus

3.2.1 Syntax

The set of MCP processes is defined by the following grammar, where *schema* is a JSON Schema object and *uri* a resource identifier[2]:

[2]With the latest release of the MCP specification the protocol is sessionless (see: "SEP-2567: Sessionless MCP via Explicit State Handles").

$$
\begin{array}{lll}
M & ::= & \mathsf{Tool}\langle n, d, \mathit{schema}\rangle & \text{tool definition} \\
& | & \mathsf{Resource}\langle \mathit{uri}, \mathit{content}\rangle & \text{read-only resource} \\
& | & \mathsf{Prompt}\langle \mathit{template}, \mathit{args}\rangle & \text{workflow template} \\
& | & \mathsf{ToolsList}\langle \mathit{metadata}\rangle & \text{discovery response} \\
& | & \mathsf{ToolCall}\langle n, \mathit{params}\rangle & \text{tool invocation} \\
& | & \mathsf{Validate}\langle \mathit{params}, \mathit{schema}\rangle & \text{parameter validation} \\
& | & \mathsf{Execute}\langle n, \mathit{params}\rangle & \text{tool execution} \\
& | & \mathsf{Result}\langle \mathit{output}\rangle & \text{return value} \\
& | & \mathsf{Error}\langle \mathit{type}, \mathit{msg}\rangle & \text{error state} \\
& | & M_1 \,|\, M_2 & \text{parallel composition} \\
& | & (\nu\, c)\, M & \text{channel restriction} \\
& | & !M & \text{replication} \\
& | & \mathbf{0} & \text{null process}
\end{array}
$$

3.2.2 Example: Train Booking in MCP

This example invokes the tool `bookTrainTool`. In MCP the fact that the invocation needs a confirmation by the user and that `class` is an optional parameter can only be expressed in the description text.

```
BookTrainTool = Tool<
  name = "book_train",
  description = "Books a train ticket, needs confirmation",
  schema = {
    "type": "object",
    "required": ["from", "to", "date"],
    "properties": {
      "from": {"type": "string", "description": "Origin Code"},
      "to": {"type": "string", "description": "Destination Code"},
      "date": {"type": "string", "description": "Date of Travel"},
      "class": {"type": "string", "description": "Class (default: economy)"}
    }
  }
>
```

3.2.3 Operational Semantics

The transition rules for MCP processes are given in Figure 2.

3.3 Labeled Transition Systems

For both SGD and MCP we define a labeled transition system. In both systems, the parameter n denotes the tool/intent name, p denotes the parameter, t denotes the error type, m the error message and o the object type.

$$\frac{}{\mathsf{ToolsList}\langle \mathit{tools} \rangle \xrightarrow{\mathsf{list}(f)} \{t \in \mathit{tools} \mid \mathsf{matches}(t, f)\}}\ \textsc{MCP-Discover}$$

$$\frac{}{\mathsf{Tool}\langle n, d, \mathit{schema} \rangle \xrightarrow{\mathsf{call}(n, \mathit{params})} \mathsf{Validate}\langle \mathit{params}, \mathit{schema} \rangle}\ \textsc{MCP-Call}$$

$$\frac{\mathsf{conforms}(\mathit{params}, \mathit{schema})}{\mathsf{Validate}\langle \mathit{params}, \mathit{schema} \rangle \xrightarrow{\tau} \mathsf{Execute}\langle n, \mathit{params} \rangle}\ \textsc{MCP-Validate-Ok}$$

$$\frac{\neg\,\mathsf{conforms}(\mathit{params}, \mathit{schema})}{\mathsf{Validate}\langle \mathit{params}, \mathit{schema} \rangle \xrightarrow{\tau} \mathsf{Error}\langle \mathsf{ValidationError}, \mathit{violations} \rangle}\ \textsc{MCP-Validate-Err}$$

$$\frac{\mathit{output} = \mathsf{invoke\_tool}(n, \mathit{params})}{\mathsf{Execute}\langle n, \mathit{params} \rangle \xrightarrow{\mathsf{execute}} \mathsf{Result}\langle \mathit{output} \rangle}\ \textsc{MCP-Execute}$$

$$\frac{}{\mathsf{Resource}\langle \mathit{uri}, \mathit{content} \rangle \xrightarrow{\mathsf{read}(\mathit{uri})} \mathsf{Result}\langle \mathit{content} \rangle}\ \textsc{MCP-Resource}$$

Figure 2: Operational semantics for MCP processes.

Definition 3.1 (SGD LTS)**.** The labeled transition system for SGD is the tuple $(\mathit{States}_{\mathsf{SGD}}, \mathit{Labels}_{\mathsf{SGD}}, \rightarrow_{\mathsf{SGD}})$ where:

$$\begin{aligned} \mathit{States}_{\mathsf{SGD}} &= \{S \mid S \text{ is a well-formed SGD process}\} \\ \mathit{Labels}_{\mathsf{SGD}} &= \{\mathsf{invoke}(n, p),\ \mathsf{collect}(s, v), \\ &\qquad \mathsf{execute},\ \mathsf{result}(o),\ \mathsf{error}(t, m)\} \\ \rightarrow_{\mathsf{SGD}} &\subseteq \mathit{States}_{\mathsf{SGD}} \times \mathit{Labels}_{\mathsf{SGD}} \times \mathit{States}_{\mathsf{SGD}} \end{aligned}$$

where s denotes the slot and v denotes the values.

Definition 3.2 (MCP LTS)**.** The labeled transition system for MCP is the tuple $(\mathit{States}_{\mathsf{MCP}}, \mathit{Labels}_{\mathsf{MCP}}, \rightarrow_{\mathsf{MCP}})$ where:

$$\begin{aligned} \mathit{States}_{\mathsf{MCP}} &= \{M \mid M \text{ is a well-formed MCP process}\} \\ \mathit{Labels}_{\mathsf{MCP}} &= \{\mathsf{call}(n, p),\ \mathsf{list}(f),\ \tau,\ \mathsf{execute}, \\ &\qquad \mathsf{result}(o),\ \mathsf{error}(t, m),\ \mathsf{read}(u)\} \\ \rightarrow_{\mathsf{MCP}} &\subseteq \mathit{States}_{\mathsf{MCP}} \times \mathit{Labels}_{\mathsf{MCP}} \times \mathit{States}_{\mathsf{MCP}} \end{aligned}$$

where u denotes a universal resource identifier (URI).

Silent (τ) actions are $\mathsf{collect}(s, v)$ and validate. Observable actions are $\mathsf{invoke}(n, p)$, $\mathsf{call}(n, p)$, execute, $\mathsf{result}(\cdot)$, $\mathsf{error}(\cdot, \cdot)$, and $\mathsf{read}(\cdot)$.

4 Forward Bisimulation: SGD → MCP

4.1 Mapping Function Φ

Definition 4.1 (Schema Mapping $\Phi : \mathsf{SGD} \to \mathsf{MCP}$)**.**

$$
\begin{aligned}
\Phi(\mathsf{Intent}\langle n, d, R, O, t\rangle) &= \mathsf{Tool}\langle n, d, \mathsf{schema}(R, O)\rangle \\
\text{where} \quad \mathsf{schema}(R, O) &= \Big\{\text{"}\texttt{type}\text{": "}\texttt{object}\text{", "}\texttt{required}\text{": } [s.\mathit{name} \mid s \in R], \\
&\qquad \text{"}\texttt{properties}\text{": } \{s.\mathit{name} : \mathsf{prop}(s) \mid s \in R \cup O\}\Big\} \\
\mathsf{prop}(s) &= \{\text{"}\texttt{type}\text{": } s.\mathit{type}, \text{"}\texttt{description}\text{": } s.\mathit{desc}\} \\
&\quad \cup (\{\text{"}\texttt{enum}\text{": } s.\mathit{vals}\} \text{ if } s.\mathit{vals} \neq \emptyset) \\
\Phi(\mathsf{Execute}\langle n, \mathit{bindings}\rangle) &= \mathsf{ToolCall}\langle n, \mathsf{JSON.encode}(\mathit{bindings})\rangle \\
\Phi(\mathsf{Result}\langle o\rangle) &= \mathsf{Result}\langle o\rangle \\
\Phi(\mathsf{Error}\langle t, m\rangle) &= \mathsf{Error}\langle t, m\rangle \\
\Phi(S_1 \mid S_2) &= \Phi(S_1) \mid \Phi(S_2) \\
\Phi((\nu\, c)\, S) &= (\nu\, c)\, \Phi(S) \\
\Phi(!S) &= !\Phi(S) \\
\Phi(\mathbf{0}) &= \mathbf{0}
\end{aligned}
$$

4.2 Forward Mapping Examples

4.2.1 Φ Mapping of Train Intent

```
SGD_Train = Intent<
  "BookTrain",
  "Books a Train",
  [Slot<"from", "string", ["8507000", "8508050"]>],
  [],
  true
>

phi(SGD_Train) = Tool<
  "BookTrain",
  "Books a Train",
  {
    "type": "object",
    "required": ["from"],
    "properties": {
      "from": {
        "type": "string",
        "description": "Departure station",
        "enum": ["8507000", "8508050"]
      }
    }
  }
```

>

4.3 Action Equivalence

Definition 4.2 (Action Equivalence $\approx$)**.**

$$\begin{array}{ccc} \mathsf{invoke}(n,p) \approx \mathsf{call}(n,p), & \mathsf{list}(f) \approx \mathsf{list}(f), & \mathsf{collect}(s,v) \approx \tau, \\ \mathsf{execute} \approx \mathsf{execute}, & \mathsf{result}(o) \approx \mathsf{result}(o), & \mathsf{error}(t,m) \approx \mathsf{error}(t,m) \end{array}$$

4.4 Bisimulation and Main Theorem

We write $S \sim M$ iff there exists a *weak bisimulation* $\mathscr{R}$ with $(S,M) \in \mathscr{R}$.

Theorem 4.3 (SGD $\sim$ MCP under Φ)**.** $\forall S \in \mathsf{SGD}.\ S \sim \Phi(S)$.

Proof. Define $\mathscr{R} = \{(S, \Phi(S)) \mid S \in \mathsf{SGD}\}$. We prove $\mathscr{R}$ is a weak bisimulation by structural induction.

Lemma 4.4 (Intent $\sim$ Tool)**.** $\mathsf{Intent}\langle n, d, R, O, t\rangle \ \sim\ \mathsf{Tool}\langle n, d, \mathsf{schema}(R,O)\rangle$.

Proof. Suppose $\mathsf{Intent}\langle n, d, R, O, t\rangle \xrightarrow{\mathsf{invoke}(n,p)} S'$.

Case 1: p satisfies all slots in R. Then $S' = \mathsf{Execute}\langle n, p\rangle$. For the tool:

$$\mathsf{Tool}\langle n, d, \mathit{schema}\rangle \xrightarrow{\mathsf{call}(n,p)} \mathsf{Validate}\langle p, \mathit{schema}\rangle \xrightarrow{\tau} \mathsf{Execute}\langle n, p\rangle$$

Since $\mathsf{schema}(R,O).\texttt{required} = [s.\mathit{name} \mid s \in R]$, validation succeeds iff p satisfies R. By definition, $\mathsf{Execute}\langle n, p\rangle \sim \mathsf{Execute}\langle n, p\rangle$. ✓

Case 2: p missing required slots. Then $S' = \mathsf{Error}\langle \mathsf{MissingSlots}, R \setminus p\rangle$. The tool produces $\mathsf{Error}\langle \mathsf{ValidationError}, R \setminus p\rangle$ via the MCP-VALIDATE-ERR rule; error terms are observationally equivalent. ✓

The backward direction is symmetric by construction of Φ. □

Equivalently, S and $\Phi(S)$ produce the same observable traces up to τ-absorption, so any execution sequence demonstrating safe behaviour in SGD lifts to MCP and vice versa.

Lemma 4.5 (Execution Preservation)**.**

$$\mathsf{Execute}\langle n, \mathit{bindings}\rangle_{\mathsf{SGD}} \ \sim\ \mathsf{Execute}\langle n, \mathsf{JSON.encode}(\mathit{bindings})\rangle_{\mathsf{MCP}}.$$

Proof. Both processes transition via execute to $\mathsf{Result}\langle o\rangle$ where $o = \mathsf{invoke\_api}(n, \mathit{bindings})$. JSON encoding is a bijection on well-typed parameters, so

$$\mathsf{invoke\_api}(n,b) = \mathsf{invoke\_api}(n, \mathsf{decode}(\mathsf{encode}(b))).$$

□

Induction.
Base case ($S = \mathbf{0}$): $\Phi(\mathbf{0}) = \mathbf{0}$ and $\mathbf{0} \sim \mathbf{0}$ trivially.
Inductive case ($S = S_1 \mid S_2$): By IH, $S_i \sim \Phi(S_i)$. If $S_1 \mid S_2 \xrightarrow{\alpha} S_1' \mid S_2$ then by IH $\Phi(S_1) \xrightarrow{\beta} M_1'$ with $S_1' \sim M_1'$ and $\alpha \approx \beta$, hence $\Phi(S_1) \mid \Phi(S_2) \xrightarrow{\beta} M_1' \mid \Phi(S_2)$, and $(S_1' \mid S_2) \sim (M_1' \mid \Phi(S_2))$. The symmetric case for S_2 is identical.
Inductive case ($S = \mathsf{Intent}\langle n, d, R, O, t\rangle$): By Lemma 4.4.
Inductive case ($S = (\nu\, c)\, S'$): $\Phi((\nu\, c)\, S') = (\nu\, c)\, \Phi(S')$. By IH, $S' \sim \Phi(S')$. Channel restriction preserves bisimulation (standard π-calculus result [14]). □

Corollary 4.6 (Zero-Shot Generalization Transfer)**.** *If an* SGD *model handles Intent I zero-shot via schema s, then an* MCP *model handles* $\mathsf{Tool}\langle \cdot, \cdot, \Phi(s)\rangle$ *zero-shot via* $\Phi(s)$.

5 The Reverse Direction: A Gorla-Style Analysis

Section 4 exhibited a mapping Φ and proved $S \sim \Phi(S)$. We now ask the question this raises in the language of encodability: *is Φ a good encoding, and does a good encoding run the other way?* We adopt Gorla's criteria for language encodings [10] and show that Φ is a *valid* encoding which is, however, neither *surjective* nor *fully abstract*. These two failures are precisely the gaps that Sections 6–7 close.

5.1 Encoding Criteria

Definition 5.1 (Valid encoding [10])**.** An encoding $(\llbracket\cdot\rrbracket, \varphi)$ of source calculus $\mathscr{L}_s$ into target $\mathscr{L}_t$, with translation $\llbracket\cdot\rrbracket$ and renaming policy φ, is *valid* if:

Compositionality C_1 For every operator op there is a target context $\mathscr{C}^{\mathsf{op}}[\cdot_1, \ldots, \cdot_k]$ with $\llbracket \mathsf{op}(P_1, \ldots, P_k)\rrbracket = \mathscr{C}^{\mathsf{op}}[\llbracket P_1\rrbracket, \ldots, \llbracket P_k\rrbracket]$.

Name invariance C_2 $\llbracket\sigma(P)\rrbracket = \sigma'(\llbracket P\rrbracket)$ for σ' induced by σ through φ.

Operational correspondence C_3 *Completeness:* $P \Longrightarrow_s P'$ implies $\llbracket P\rrbracket \Longrightarrow_t \sim \llbracket P'\rrbracket$. *Soundness:* $\llbracket P\rrbracket \Longrightarrow_t T$ implies $\exists P'.\ P \Longrightarrow_s P'$ and $T \Longrightarrow_t \sim \llbracket P'\rrbracket$.

Divergence reflection C_4 $\llbracket P\rrbracket$ diverges only if P diverges.

Success sensitiveness C_5 $P \Downarrow_s \checkmark$ iff $\llbracket P\rrbracket \Downarrow_t \checkmark$ for a success barb $\checkmark$.

Definition 5.2 (Full abstraction)**.** A valid encoding is *fully abstract* if it preserves and reflects the source equivalence: $P \sim_s Q \iff \llbracket P\rrbracket \sim_t \llbracket Q\rrbracket$.

5.2 Φ is a Valid Encoding

Proposition 5.3 (Validity of Φ)**.** (Φ, id) *is a valid encoding of* SGD *into* MCP.

Proof. **(C_1)** Definition 4.1 is homomorphic on the operators ($\Phi(S_1 \mid S_2) = \Phi(S_1) \mid \Phi(S_2)$, and likewise for (νc), !, $\mathbf{0}$); the witnessing contexts are the operators themselves, name-independent. **(C_2)** Φ copies the name n through and never branches on name identity; with $\varphi = \mathrm{id}$, invariance is immediate. **(C_3)** Theorem 4.3 ($S \sim \Phi(S)$) is an operational correspondence up to $\sim$. **(C_4)** The only τ-steps Φ introduces are the administrative validate transitions, one per invocation (MCP-VALIDATE-OK); they cannot chain, so no divergence is added. **(C_5)** With $\checkmark$ the result barb, $S \sim \Phi(S)$ and barb-respect of $\sim$ give the equivalence. □

Φ is thus exactly the structure-preserving translation the encodability literature certifies. The rest of the section identifies the two ways it falls short of an equivalence of calculi.

5.3 Φ is Not Surjective

The reverse map Φ^{-1} is defined only on the image of Φ. On the executable Tool fragment it is the expected inverse; on the remaining primitives it is undefined.

Definition 5.4 (Partial inverse Φ^{-1})**.**

$$\Phi^{-1}(\mathsf{Tool}\langle n, d, \mathit{schema}\rangle) = \mathsf{Intent}\langle n, d, R(\mathit{schema}), O(\mathit{schema}), ?\rangle,$$

with

$$R(\mathit{schema}) = \{\mathsf{Slot}\langle x, \mathit{schema}.\mathrm{prop}[x], \mathit{enum}\rangle \mid x \in \mathit{schema}.\mathrm{required}\},$$
$$O(\mathit{schema}) = \{\mathsf{Slot}\langle y, \mathit{schema}.\mathrm{prop}[y], \mathit{enum}\rangle \mid y \in \mathit{schema}.\mathrm{properties} \setminus \mathit{schema}.\mathrm{required}\},$$

where *enum* denotes the values from the JSON schema's enum field (or $\emptyset$ if absent). The flag t is left undetermined (?), and Φ^{-1} is undefined on Resource, Prompt, and ToolsList.

Theorem 5.5 (Non-surjectivity)**.** $\mathrm{Image}(\Phi) \subsetneq \mathsf{MCP}$. *In particular* $\mathsf{Resource}\langle\cdot,\cdot\rangle$, $\mathsf{Prompt}\langle\cdot,\cdot\rangle$, *and the discovery process* $\mathsf{ToolsList}\langle\cdot\rangle$ *lie outside* $\mathrm{Image}(\Phi)$.

Proof. By Definition 4.1, Φ produces only Tool terms under the closure operators; Resource, Prompt, and ToolsList are none of these, so no S has $\Phi(S)$ headed by them. □

Remark 5.6 (An image fact, not a separation)**.** Theorem 5.5 says these primitives are absent from the *image of* Φ, not that they are inexpressible in SGD. We make no separation claim: with replication and channel mobility, SGD admits degenerate encodings of both (a passive resource as an empty-slot intent returning its content; discovery via input-guarded choice). The point is structural – these are primitives SGD lacks *as primitives* – so the canonical inverse Φ^{-1}, which must respect primitive structure, cannot be total. They are surplus to be *quarantined* (Section 7), not gaps to be closed.

5.4 Φ is Not Fully Abstract

The substantive gap is not non-surjectivity but the failure of Φ to *reflect* behavioural equivalence: Φ collapses SGD distinctions that MCP cannot observe.

Theorem 5.7 (Φ does not reflect $\sim$)**.** *There exist* $S_1, S_2 \in \mathsf{SGD}$ *with* $S_1 \not\sim S_2$ *and* $\Phi(S_1) \sim \Phi(S_2)$. *Hence* Φ *is not fully abstract.*

Proof. Let $S_1 = \mathsf{Intent}\langle n, d, R, O, \mathsf{true}\rangle$ and $S_2 = \mathsf{Intent}\langle n, d, R, O, \mathsf{false}\rangle$, equal but for t. They are distinguished by the approval gate: by SGD-EXECUTE-TX, S_1 requires approval before execute, whereas S_2 proceeds via SGD-EXECUTE without it. Under any equivalence observing the approval interaction – made explicit at the process level by the prefix $\mathsf{approval?}\langle \mathit{confirm}\rangle$ of Section 6.2 – we have $S_1 \not\sim S_2$. Yet $\Phi(S_1) = \mathsf{Tool}\langle n, d, \mathit{schema}(R, O)\rangle = \Phi(S_2)$: the flag t is carried by no field of *schema*, surviving at best as prose in a description string that MCP's rules do not interpret. Identical terms are bisimilar, so $\Phi(S_1) \sim \Phi(S_2)$. □

The collapse is concrete. A tool that needs user approval before reading an account loses precisely that requirement on the way back to SGD:

```
M_1 = Tool<
  "fetch_user",
  "fetch a user's data, needing its approval",
  { "type": "object", "required": ["user_id"],
    "properties": { "user_id": {"type": "string"} } }
>

Phi^{-1}(M_1) = Intent<
  "fetch_user", "fetch a user's data, needing its approval",
  [Slot<"user_id", "string", []>], [],
  ?   // UNDEFINED: is_transactional cannot be recovered
>
```

The is_transactional flag is critical for safety (it gates user approval) but is absent from MCP schemas. Inference from the description string is unreliable and not compositionally verifiable.

Corollary 5.8 (Lossy round trip, non-injective inverse)**.** $\Phi^{-1} \circ \Phi \neq \mathrm{id}_{\mathsf{SGD}}$*, and* Φ^{-1} *is not injective where defined.*

Proof. The round trip drops t: starting from a transactional intent,

```
S = Intent<"book_ticket", ..., ..., ..., true>

Phi(S) = Tool<"book_ticket", ..., schema>
         (is_transactional lost)

Phi^{-1}(Phi(S)) = Intent<"book_ticket", ..., ..., ..., ?>
                   != S
```

so $\Phi^{-1} \circ \Phi \neq \mathrm{id}$; the lost information includes the transactionality flag, error-recovery strategies, and tool dependencies encoded in dialogue flow. Dually, two tools differing only in description – hence in implicit side effects MCP does not structurally parse – have the same image under Φ^{-1}:

```
M_1 = Tool<"book_ticket", "Books a ticket", schema>

M_2 = Tool<"book_ticket",
           "Books a ticket [SIDE EFFECT: sends email]",
           schema>   // same schema, different implicit side effects

Phi^{-1}(M_1) = Phi^{-1}(M_2)
              = Intent<"book_ticket", ..., is_transactional=?>
```

Since Φ^{-1} does not parse descriptions structurally, distinct tools collapse to one intent. □

Remark 5.9 (What is collapsed)**.** Theorem 5.7 isolates one collapsed coordinate, the flag t. The same failure recurs for every SGD datum that MCP records only as prose: error-recovery behaviour and inter-tool ordering are observable in SGD executions yet invisible to MCP's rules. Each collapsed coordinate is a distinction Φ fails to reflect.

5.5 Diagnosis: Exactly Four Coordinates

The two failures have different remedies. Non-surjectivity (Section 5.3) is benign: the surplus primitives are restricted away, leaving the executable fragment MCP^+ of Section 7. Non-full-abstraction (Section 5.4) is the failure that demands new structure: for Φ to reflect $\sim$, every SGD distinction must become a machine-readable, operationally interpreted coordinate of the target. There are exactly four collapsed coordinates, giving the four principles of Section 6:

P1 (semantic completeness) makes the description recoverable as slot semantics, so Φ^{-1} reconstructs slots reliably rather than parsing prose;

P2 (explicit action boundaries) makes t a first-class observable (requires_approval), repairing exactly the collapse of Theorem 5.7 and Corollary 5.8;

P3 (failure mode documentation) makes error-recovery branches observable, so behavioural equivalence holds on error paths;

P4 (inter-tool relationship declaration) makes dialogue-flow dependencies explicit, so tool sequencing is not lost.

Section 7 shows these four extensions make Φ^+ fully abstract on the executable fragment, and that they are *minimal* – dropping any one reinstates a collapsed coordinate and breaks reflection – yielding the fully abstract bijection $\mathsf{SGD} \cong \mathsf{MCP}^+$.

6 Four Principles as Type-System Extensions

We formalize four design principles as type-system extensions to close the gaps identified in the previous section.

6.1 Principle 1: Semantic Completeness

Informal: Descriptions must convey *why* parameters exist, not merely their types.

Definition 6.1 (Semantic Density)**.**

$$\mathsf{sem\_density}(s) \;=\; \frac{|\mathsf{entities}(s)|}{|\mathsf{tokens}(s)|}$$

where $\mathsf{entities}(s)$ counts named entities, examples, and constraints in string s, and θ is a threshold (e.g., 0.2).

Type Rule (P1):

$$\frac{\Gamma \vdash \mathit{desc} : \mathsf{String} \quad \mathsf{sem\_density}(\mathit{desc}) \geq \theta}{\Gamma \vdash \{\texttt{description}\colon \mathit{desc}\} : \mathsf{WellDescribed}}\ \textsc{WellDescribed}$$

6.1.1 Principle P_1: Semantic Completeness Examples

Violates P1:

```
"departure": {"type": "string", "description": "departure"}
// sem_density = 0 (no new information)
```

Satisfies P1:

```
"departure": {
  "type": "string",
  "description": "UIC station code (e.g., 8507000, 8508050)"
}
// sem_density = 2/10 = 0.2, includes concept + 2 examples
```

6.2 Principle P_2: Explicit Action Boundaries

Informal: Tools must signal if invocation causes side effects: side_effects := read | write | delete | none.
Type Rule (P2):

$$\frac{\mathit{meta}.\mathsf{side\_effects} \in \{\mathsf{write}, \mathsf{delete}\}}{\mathit{meta}.\mathsf{requires\_approval} = \mathsf{true}}\ \textsc{WriteApproval}$$

The process-level encoding of a write-capable tool is:

$$\begin{aligned}\mathsf{Tool}_{\mathsf{write}} \;=\; & \mathsf{call}?\langle \mathit{params}\rangle \,.\, \mathsf{approval}?\langle \mathit{confirm}\rangle. \\ & \big(\mathit{confirm} = \mathsf{true} \;\to\; \mathsf{execute}\langle \mathit{params}\rangle \,.\, \mathsf{result}!\langle \mathsf{ok}\rangle \,.\, \mathbf{0} \\ & \mid \mathit{confirm} = \mathsf{false} \;\to\; \mathsf{result}!\langle \mathsf{cancelled}\rangle \,.\, \mathbf{0}\big)\end{aligned}$$

Safety as well as privacy can both be regarded as action boundaries. In both cases, a user is asked for permission before an action.

6.2.1 Principle P_2: Explicit Action Boundaries Example

```
Tool<
  "delete_user",
  "Permanently deletes a user account",
  { "type": "object", "required": ["user_id"],
    "properties": { "user_id": {"type": "string"} } },
  metadata = {
    side_effects: delete,
    requires_approval: true   // enforced by type rule WriteApproval
  }
>
```

Theorem 6.2 (No Unapproved Side Effects)**.** $\forall$ Tool *with* side_effects $\in \{\mathsf{write}, \mathsf{delete}\}$*: in every execution trace,* execute *is preceded by* approval.

6.3 Principle P_3: Failure Mode Documentation

Informal: Enumerate expected error conditions and recovery strategies.

$$\begin{aligned}\mathsf{failure\_modes} &::= [(\mathit{ErrorType}, \mathit{RecoveryStrategy})] \\ \mathit{ErrorType} &::= \mathsf{ValidationError} \mid \mathsf{AuthError} \mid \mathsf{ServiceDown} \mid \mathsf{NotFound} \mid \cdots \\ \mathit{RecoveryStrategy} &::= \mathsf{Retry}(n) \mid \mathsf{Fallback}(\mathit{tool}) \mid \mathsf{UserPrompt}(m) \mid \mathsf{Abort}\end{aligned}$$

6.3.1 Failure Mode Documentation Example

```
Tool<
  "fetch_user_data",
  "Retrieves user information from database",
  schema,
  metadata = {
    failure_modes: [
      (NotFound,     UserPrompt("User does not exist. Create new?")),
      (ServiceDown,  Retry(3)),
      (AuthError,    Fallback("use_cached_data"))
    ]
  }
>
```

A robust tool is encoded as:

$$\begin{aligned}\mathsf{Tool}_{\mathsf{robust}} \;=\; & \mathsf{execute}?\langle \mathit{params}\rangle.\\ & \big(\mathsf{success}!\langle \mathit{output}\rangle \,.\, \mathbf{0}\\ & \mid \mathsf{error}!\langle \mathsf{NotFound}, \mathit{rec}_1\rangle \,.\, \mathit{rec}_1 \,.\, \mathbf{0}\\ & \mid \mathsf{error}!\langle \mathsf{ServiceDown}, \mathit{rec}_2\rangle \,.\, \mathit{rec}_2 \,.\, \mathbf{0}\big)\end{aligned}$$

where each rec_i corresponds to the declared recovery strategy.

6.4 Principle P_4: Inter-Tool Relationship Declaration

Informal: Express dependencies between tools explicitly.

$$\begin{aligned}\mathsf{dependencies} &::= [(\mathit{tool\_name}, \mathit{relation})]\\ \mathit{relation} &::= \mathsf{Requires} \mid \mathsf{ProducesInputFor} \mid \mathsf{ExclusiveWith}\end{aligned}$$

6.4.1 Inter-Tool Relationship Declaration Example

```
Tool<
  "process_payment",
  "Processes a payment transaction",
  schema,
  metadata = {
    dependencies: [
      ("create_order",   Requires),  // must call create_order first
      ("verify_balance", Requires)   // must verify balance
    ]
  }
>
```

The process encoding of a dependent pair (T_A, T_B) with $T_B.\mathsf{dependencies} = [(T_A, \mathsf{Requires})]$:

$$\begin{aligned}T_A \;&=\; \mathsf{execute}_A?\langle \mathit{params}\rangle \,.\, \mathsf{result}_A!\langle \mathit{id}, \{\mathsf{for}: T_B\}\rangle \,.\, \mathbf{0}\\ T_B \;&=\; \mathsf{requires}?\langle \mathit{id} \text{ from } T_A\rangle \,.\, \mathsf{execute}_B?\langle \mathit{id}, \mathit{params}\rangle \,.\, \mathsf{result}_B!\langle \mathit{data}\rangle \,.\, \mathbf{0}\end{aligned}$$

Theorem 6.3 (Dependency Safety). $\forall$ *workflow* W *composed of tools* $\{T_1,\ldots,T_n\}$*: if* $T_i.\mathsf{dependencies} = [(T_j,\mathsf{Requires})]$*, then in all valid execution traces* $\mathsf{execute}_{T_j}$ *precedes* $\mathsf{execute}_{T_i}$.

7 Extended Calculus MCP^+ and Full Equivalence

7.1 MCP^+ Syntax

MCP^+ is the executable fragment of the extended calculus. The **Resource** and **Prompt** terms, identified in §5.3 as outside $\mathrm{Image}(\Phi)$, are quarantined and not part of the fully abstract bijection. The terms of MCP^+ are generated by:

$$\begin{aligned} M^+ ::= {} & \mathsf{Tool}\langle n,d,schema\rangle^+[meta] \\ & \mid M^+ \parallel M^+ \\ & \mid (\nu c)\,M^+ \\ & \mid\ !M^+ \\ & \mid 0 \end{aligned}$$

where $[meta]$ denotes the metadata block $meta = \{\ldots(\text{unchanged})\ldots\}$. Additionally, Tool^+ carries additional metadata:

$$\begin{aligned} metadata = {} & \big\{\mathsf{side\_effects} : \{\mathsf{read},\mathsf{write},\mathsf{delete},\mathsf{none}\}, \\ & \mathsf{requires\_approval} : \mathsf{Bool}, \\ & \mathsf{failure\_modes} : [(ErrorType, RecoveryStrategy)], \\ & \mathsf{summary} : \mathsf{String}, \\ & \mathsf{dependencies} : [(tool\_name, relation)]\big\} \end{aligned}$$

7.2 Extended Mapping Φ^+

$$\begin{aligned} \Phi^+(\mathsf{Intent}\langle n,d,R,O,t\rangle) &= \mathsf{Tool}\langle n,d,\mathsf{schema}(R,O)\rangle^+[meta] \\ meta &= \big\{\mathsf{side\_effects}\text{: }(t\ ?\ \mathsf{write} : \mathsf{read}), \\ &\quad \mathsf{requires\_approval}\text{: } t, \\ &\quad \mathsf{failure\_modes}\text{: }\mathsf{extract\_errors}(Intent), \\ &\quad \mathsf{summary}\text{: }\mathsf{first\_sentence}(d), \\ &\quad \mathsf{dependencies}\text{: }\mathsf{extract\_deps}(Intent)\big\} \end{aligned}$$

$$\begin{aligned} (\Phi^+)^{-1}\big(\mathsf{Tool}\langle n,d,schema\rangle^+[meta]\big) &= \mathsf{Intent}\langle n,d,R(schema),O(schema),t\rangle \\ \text{where } t &= (meta.\mathsf{side\_effects} \in \{\mathsf{write},\mathsf{delete}\}) \quad \text{(must satisfy WriteApproval invariant:} \\ &\quad \text{if } t \text{ then } meta.\mathsf{requires\_approval} \text{ holds)} \end{aligned}$$

7.3 Main Theorem: Full Equivalence

Theorem 7.1 ($\mathsf{MCP}^+ \cong \mathsf{SGD}$). *The following hold:*

1. $\forall S \in \mathsf{SGD}.\ S \sim \Phi^+(S)$,
2. $\forall M^+ \in \mathsf{MCP}^+.\ M^+ \sim (\Phi^+)^{-1}(M^+)$,
3. $(\Phi^+)^{-1} \circ \Phi^+ = \mathrm{id}_{\mathsf{SGD}}$,
4. $\Phi^+ \circ (\Phi^+)^{-1} = \mathrm{id}_{\mathsf{MCP}^+}$.

Proof. Part 1 (Φ^+ preserves structure): Φ^+ extends Φ with metadata. By Theorem 4.3, structural bisimulation is preserved. The metadata fields preserve SGD semantics:

$$\begin{aligned} t &\leftrightarrow \textit{side\_effects},\ \textit{requires\_approval},\\ d &\leftrightarrow \textit{summary},\\ \text{errors in Intent} &\leftrightarrow \textit{failure\_modes},\\ \text{inter-tool deps} &\leftrightarrow \textit{dependencies}. \end{aligned}$$

I.e. the SGD semantics are preserved: the transactionality flag t corresponds to side_effects and requires_approval; the description d to summary; errors in the Intent to failure_modes; and inter-tool dependencies to dependencies.

Part 2 (injectivity): Assume $\Phi^+(S_1) = \Phi^+(S_2)$. Then $n_1 = n_2$, $\mathsf{schema}(R_1, O_1) = \mathsf{schema}(R_2, O_2)$ implying $R_1 = R_2$, $O_1 = O_2$, and $\mathit{meta}_1 = \mathit{meta}_2$ implying $t_1 = t_2$. Hence $S_1 = S_2$.

Part 3 (surjectivity): For any $M^+ = \mathsf{Tool}\langle n, d, \mathit{schema}\rangle^+[\mathit{meta}]$, write $\mathit{req} = \mathit{schema.required}$ and $\mathit{prp} = \mathit{schema.properties}$ for brevity. Let $R^* = (\Phi^+)^{-1}(\mathit{req})$, $O^* = (\Phi^+)^{-1}(\mathit{prp} \setminus \mathit{req})$, and $t^* = (\mathit{meta.side\_effects} \neq \mathsf{read})$. Construct $S = \mathsf{Intent}\langle n, d, R^*, O^*, t^*\rangle$. Then $\Phi^+(S) = M^+$.

Part 4 (round-trip identity): Verified by direct substitution into Definitions 4.1 and the extended mapping. □

Corollary 7.2 (Necessity and Sufficiency)**.** *The four principles are necessary and sufficient for* $\mathsf{SGD} \cong \mathsf{MCP}^+$.

Proof. Sufficiency: Theorem 7.1. Necessity: removing any of the four Principles breaks the bijection (shown in Section 5). □

The four principles are a minimal extension to MCP needed for $\mathsf{SGD} \cong \mathsf{MCP}^+$. This can be shown by necessity analysis: Remove P_1 and $(\Phi^+)^{-1}$ cannot map descriptions to SGD slot semantics reliably; zero-shot generalization fails. Remove P_2 and is_transactional is unrecoverable; $(\Phi^+)^{-1}(\mathsf{Tool}^+)$ yields $\mathsf{Intent}\langle\cdot,\cdot,\cdot,\cdot,?\rangle$; not injective. Remove P_3 and SGD error-recovery strategies cannot be encoded; behavioral equivalence fails on error paths. Finally, remove P_4 and multi-turn dialogue dependencies are lost; tool sequencing becomes implicit.

8 Conclusion

We have presented a formal semantics for agentic tool protocols, proving that SGD and MCP are structurally bisimilar under the mapping Φ (Theorem 4.3). However, Φ^{-1} is partial and lossy (Theorems 5.5 and 5.7), revealing four critical gaps in MCP's expressivity. Through bidirectional analysis we identified four principles as *necessary and sufficient* conditions for full behavioral equivalence, and formalized them as type-system extensions defining MCP^+ with $\mathsf{MCP}^+ \cong \mathsf{SGD}$ (Theorem 7.1).

As LLM agents govern financial transactions, healthcare decisions, and infrastructure management, formal verification becomes essential. A natural synthesis of this work is to combine the work on

LLMbda calculi [9] and this work on formalizing agentic communication. While the first would define the *intra-agent communication*, the latter would define the *inter-agent communication*.

References

[1] Franz Achermann (2002): *Forms, Agents and Channels – Defining Composition Abstraction with Style*. Ph.D. thesis, University of Berne.

[2] Franz Achermann, Markus Lumpe, Jean-Guy Schneider & Oscar Nierstrasz (2001): *Piccola – A Small Composition Language*. In Howard Bowman & John Derrick, editors: *Formal Methods for Distributed Processing – A Survey of Object-Oriented Approaches*, Cambridge University Press, pp. 403–426.

[3] Edoardo Allegrini, Ananth Shreekumar & Z. Berkay Celik (2026): *Formalizing the Safety, Security, and Functional Properties of Agentic AI Systems*. arXiv:2510.14133.

[4] Ankur Bapna, Gokhan Tür, Dilek Hakkani-Tür & Larry Heck (2017): *Towards Zero-Shot Frame Semantic Parsing for Domain Scaling*. In: *Proc. Interspeech 2017*, pp. 2476–2480, doi:10.21437/Interspeech.2017-518. arXiv:1707.02363.

[5] Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, Tom Henighan, Rewon Child, Aditya Ramesh, Daniel M. Ziegler, Jeffrey Wu, Clemens Winter, Christopher Hesse, Mark Chen, Eric Sigler, Mateusz Litwin, Scott Gray, Benjamin Chess, Jack Clark, Christopher Berner, Sam McCandlish, Alec Radford, Ilya Sutskever & Dario Amodei (2020): *Language Models are Few-Shot Learners*. *Advances in Neural Information Processing Systems* 33.

[6] Marco Carbone, Luís Caires & Hugo Torres Vieira (2012): *Choreographies, Logically*. In: *Int. Conf. on Concurrency Theory (CONCUR)*, *LNCS* 7454, Springer, pp. 65–79.

[7] Pedro R. D'Argenio et al. (2022): *Probabilistic Process Algebras*. In: *Handbook of Process Algebra*, Elsevier.

[8] Jacob Devlin, Ming-Wei Chang, Kenton Lee & Kristina Toutanova (2019): *BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding*. *Proc. of the 2019 Conf. of the North American Chapter of the Association for Computational Linguistics*.

[9] Zac Garby, Andrew D. Gordon & David Sands (2026): *The LLMbda Calculus: AI Agents, Conversations, and Information Flow*. arXiv:2602.20064.

[10] Daniele Gorla (2010): *Towards a unified approach to encodability and separation results for process calculi*. *Information and Computation* 208(9), pp. 1031–1053, doi:10.1016/j.ic.2010.05.002.

[11] Suman Gupta, Rana Sharma & Kumar Singh (2020): *A Fast and Robust BERT-based Dialogue State Tracker for Schema-Guided Dialogue Dataset*. CEUR-WS, Vol-2666.

[12] Kohei Honda, Vasco T. Vasconcelos & Makoto Kubo (1998): *Language Primitives and Type Discipline for Structured Communication-Based Programming*. In: *European Symposium on Programming (ESOP)*, *LNCS* 1381, Springer, pp. 122–138.

[13] Robin Milner (1989): *Communication and Concurrency*. Prentice Hall.

[14] Robin Milner (1999): *Communicating and Mobile Systems: The π-Calculus*. Cambridge University Press.

[15] Martin Odersky, Yaoyu Zhao, Yichen Xu, Oliver Bracevac & Cao Nguyen Pham (2026): *Tracking Capabilities for Safer Agents*. arXiv:2603.00991.

[16] Mark Palatucci, Dean Pomerleau, Geoffrey E. Hinton & Tom M. Mitchell (2009): *Zero-Shot Learning with Semantic Output Codes*. In: *Advances in Neural Information Processing Systems (NIPS)*, 22, pp. 1410–1418.

[17] Anand S. Rao & Michael P. Georgeff (1995): *BDI Agents: From Theory to Practice*. In: *Int. Conf. on Multi-Agent Systems (ICMAS)*, pp. 312–319.

[18] Abhinav Rastogi, Xiaoxue Zang, Srinivas Sunkara, Raghav Gupta & Pranav Khaitan (2020): *Towards scalable multi-domain conversational agents: The schema-guided dialogue dataset*. In: *Proc. of the AAAI Conf. on Artificial Intelligence*, 34, pp. 8689–8696.

[19] Andreas Schlapbach (2003): *Enabling White-Box Reuse in a Pure Composition Language*.

[20] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser & Illia Polosukhin (2017): *Attention is All You Need*. *Advances in Neural Information Processing Systems*.

[21] Vaughn Vernon (2013): *Implementing Domain-Driven Design*. Addison-Wesley Professional.

[22] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Fei Ichien, Ed H. Chi, Quoc V. Le & Denny Zhou (2022): *Chain-of-Thought Prompting Elicits Reasoning in Large Language Models*. *Advances in Neural Information Processing Systems* 35.

[23] Jason D. Williams, Antoine Raux & Matthew Henderson (2016): *The Dialog State Tracking Challenge Series: A Review*. *Dialogue and Discourse* 7(3), pp. 4–33.

[24] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik R. Narasimhan & Yuan Cao (2023): *ReAct: Synergizing Reasoning and Acting in Language Models*. In: *Proc. of the 11th Int. Conf. on Learning Representations*.